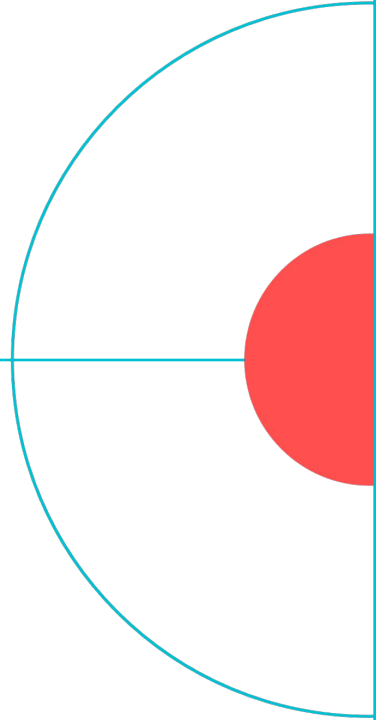
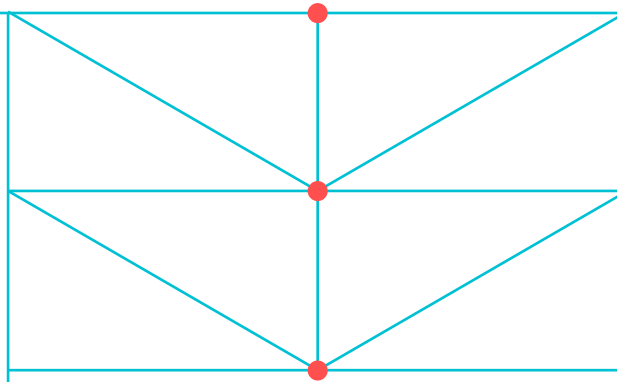


Group Testing: Finding a Few Infected Among Many

Sargunpreet Kaur

TUHH
Technische
Universität
Hamburg



Seminar: Randomized Algorithms for
Distributed Systems
Institute of Data Engineering

Why Test Individuals one by one?

- Large populations need to be screened
- Only a **small fraction** of individuals are infected
- Individual testing requires **one test per person**
- This is **slow, costly, and inefficient** when infections are rare

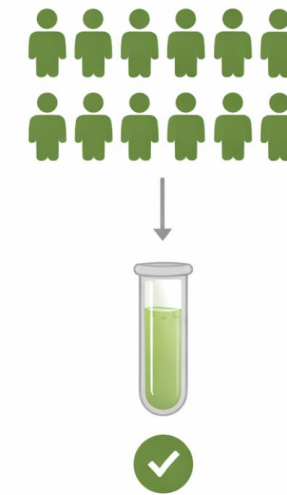


Most individuals are healthy; only a few are infected.

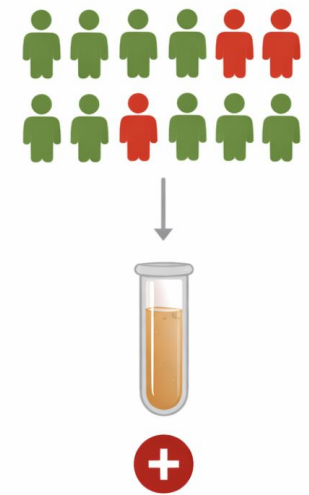
Key question:

Can we identify the infected individuals using far fewer tests?

- Instead of testing individuals one by one, we **test groups of people together**
- Each test is performed on a **pool of samples**
- The test result is:
 - **Negative** → everyone in the group is healthy
 - **Positive** → at least one person in the group is infected



Negative
Everyone is healthy



Positive
At least one person is infected

Group testing exploits the fact that infections are rare.

Key idea:

A single test can provide information about many individuals at once.

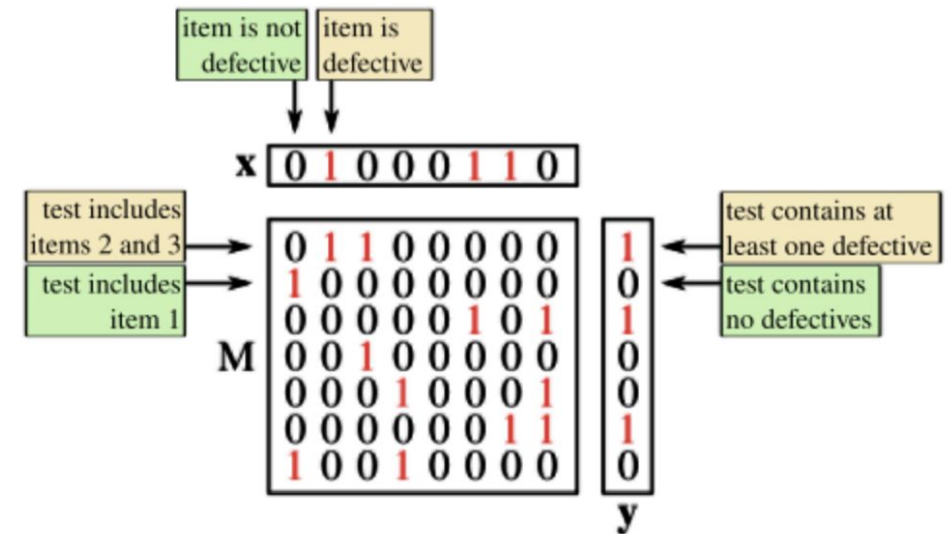
- All tests are **designed in advance** and performed **in parallel**
- Each test returns only a **binary outcome**
- From these outcomes, we must determine **which individuals are infected**

Key challenge:

Many possible infection patterns fit the same test results.

Remark:

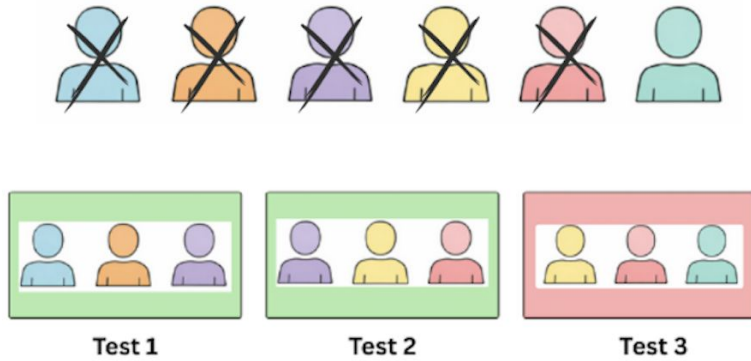
While adaptive testing is also studied, we focus on the non-adaptive setting because it captures parallel testing scenarios and admits sharp information-theoretic and algorithmic guarantees.



Each test includes multiple individuals, and each individual appears in multiple tests.

The Definite Defectives (DD) Algorithm

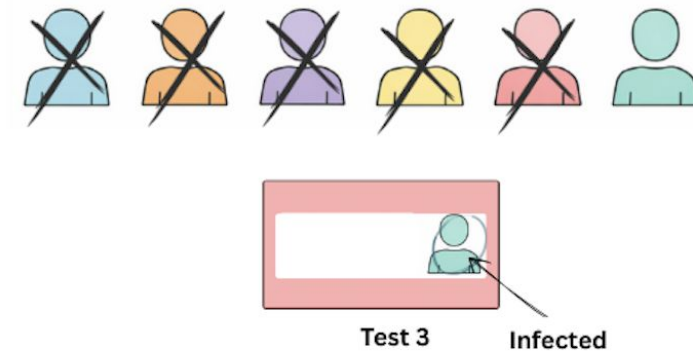
Stage 1: Elimination (Who is definitely NOT defective?)



Negative tests eliminate all items they contain.

- If a test is **negative**, all its items are **non-defective**
- Eliminate all such items
- Remaining items = **possible defectives**

Stage 2: Isolation (Who is definitely defective?)



An item is declared defective if it appears alone in a positive test.

- Look at **positive tests only**
- If a test contains **exactly one remaining item**
- That item is **definitely defective**

Bernoulli Test Design (Unbalanced)

We represent the tests by a binary matrix

$$M \in \{0, 1\}^{T \times n}$$

$$M_{t,i} \sim \text{Bernoulli}(p), \quad \text{independently}$$

$$p = \frac{\nu}{k}$$

where:

- k number of infected individuals
- ν : expected number of infected per test (constant)

Each entry means:

- $M_{t,i} = 1$: individual i participates in test t
- $M_{t,i} = 0$: individual i is not included

This gives:

$$\mathbb{E}[\text{infected per test}] = \nu$$

Test outcomes

Each test returns:

$$Y_t = \bigvee_{i \in [n]} (M_{t,i} \wedge X_i)$$

- $Y_t = 0$: all individuals in test t are healthy
- $Y_t = 1$: at least one infected individual is present

Probability a test is negative:

$$\Pr(Y_t = 0) \approx e^{-\nu}$$

Why DD Algo uses Bernoulli Test Design:

- **Many negative tests** → strong elimination of healthy individuals
- **Random inclusion** → chance of isolating infected individuals
- **Simple & analyzable** → sharp algorithmic guarantees

With $\nu = 1$:

$$p = \frac{1}{k}, \quad \Pr(Y_t = 0) \approx e^{-1}$$

This balances:

- elimination (Stage 1 of DD)
- isolation (Stage 2 of DD)

Near-Constant Test Design (Balanced)

Let:

- T = total number of tests
- n = number of individuals

For each individual i :

$$\sum_{t=1}^T M_{t,i} \approx d$$

where:

- d is fixed (or tightly concentrated)
- typically $d \approx \frac{T}{k}$

So:

- No individual appears "too little"
- No individual appears "too often"

This contrasts with Bernoulli, where appearances are random and uneven.

Test composition

In balanced designs:

- Each test contains roughly the same number of individuals
- Each individual appears in roughly the same number of tests

But:

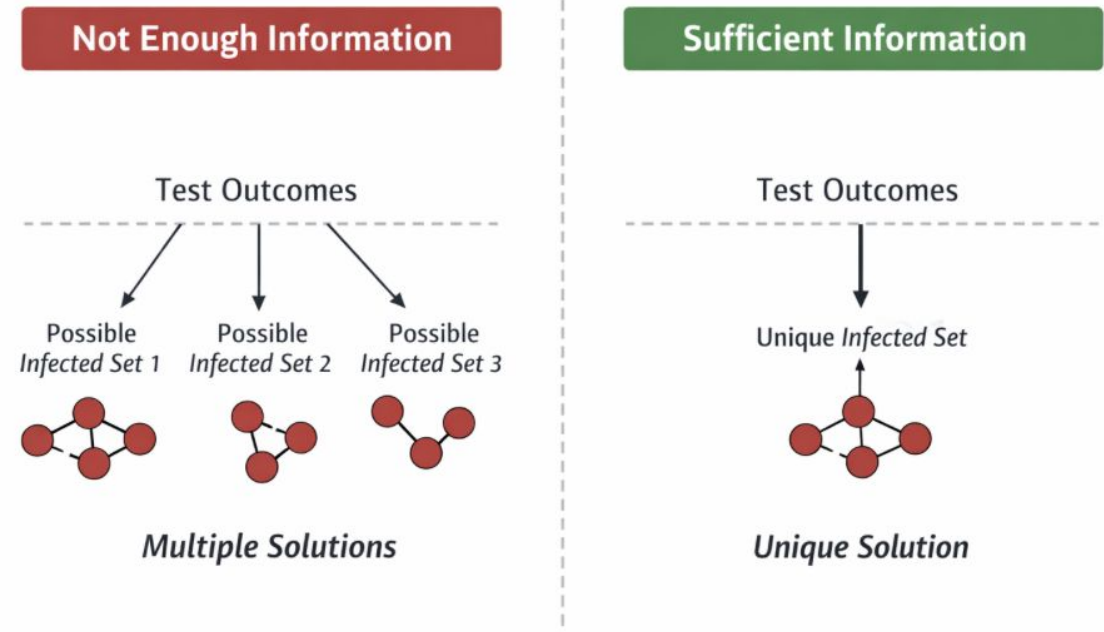
- **Which tests** an individual appears in is still random

So the design is:

random but controlled

What Determines Whether Recovery Is Unique?

1. **Number of Tests** — Amount of Information
2. **Test Design** — Distribution of Information
3. **Sparsity** — Quality of Information



Too few tests: multiple infection patterns fit the same outcomes.

Key insight:

There is a critical number of tests at which the test outcomes contain **enough information** to uniquely identify the infected set.

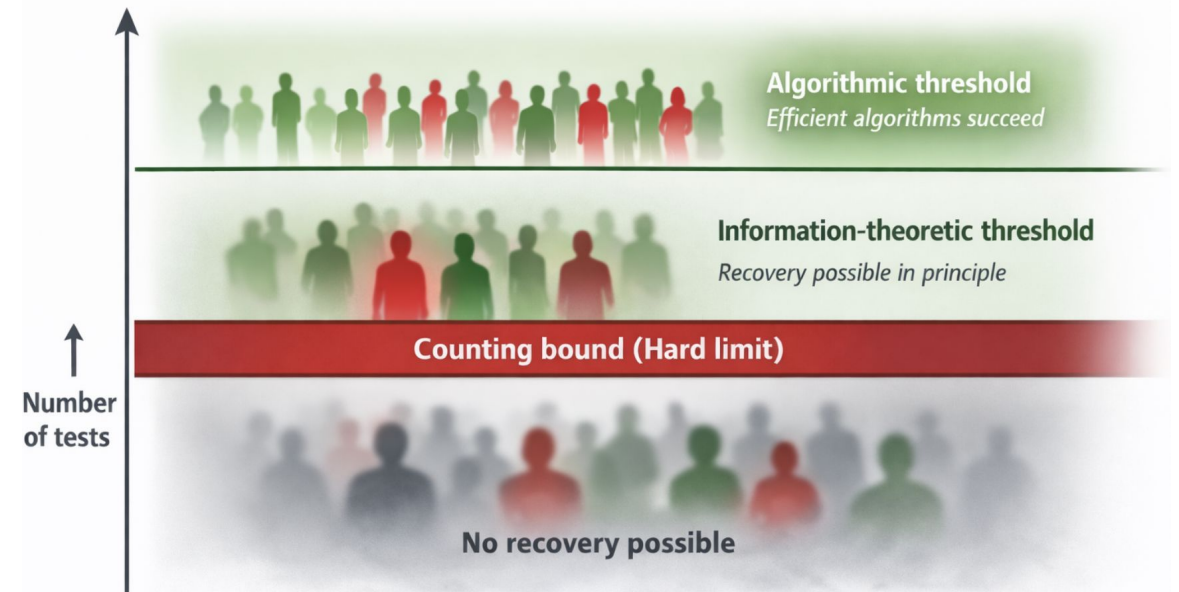
- **Counting Bound** — *Impossibility*

$$2^m \geq \binom{n}{k}$$

- n = total population
- k = infected people
- m = number of tests

- **Information-Theoretic Threshold** — *Possibility*

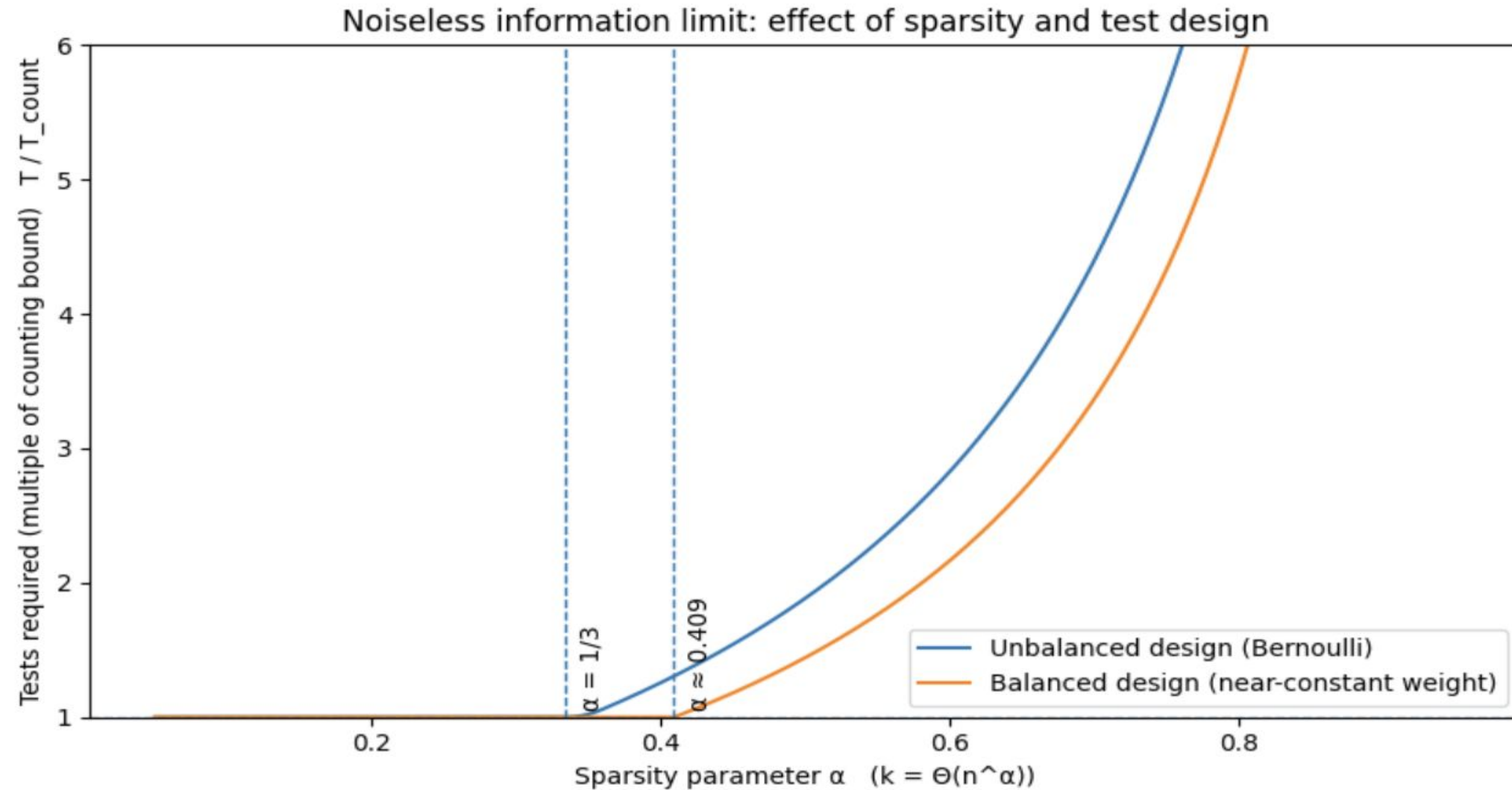
- **Algorithmic Threshold** — *Efficiency*

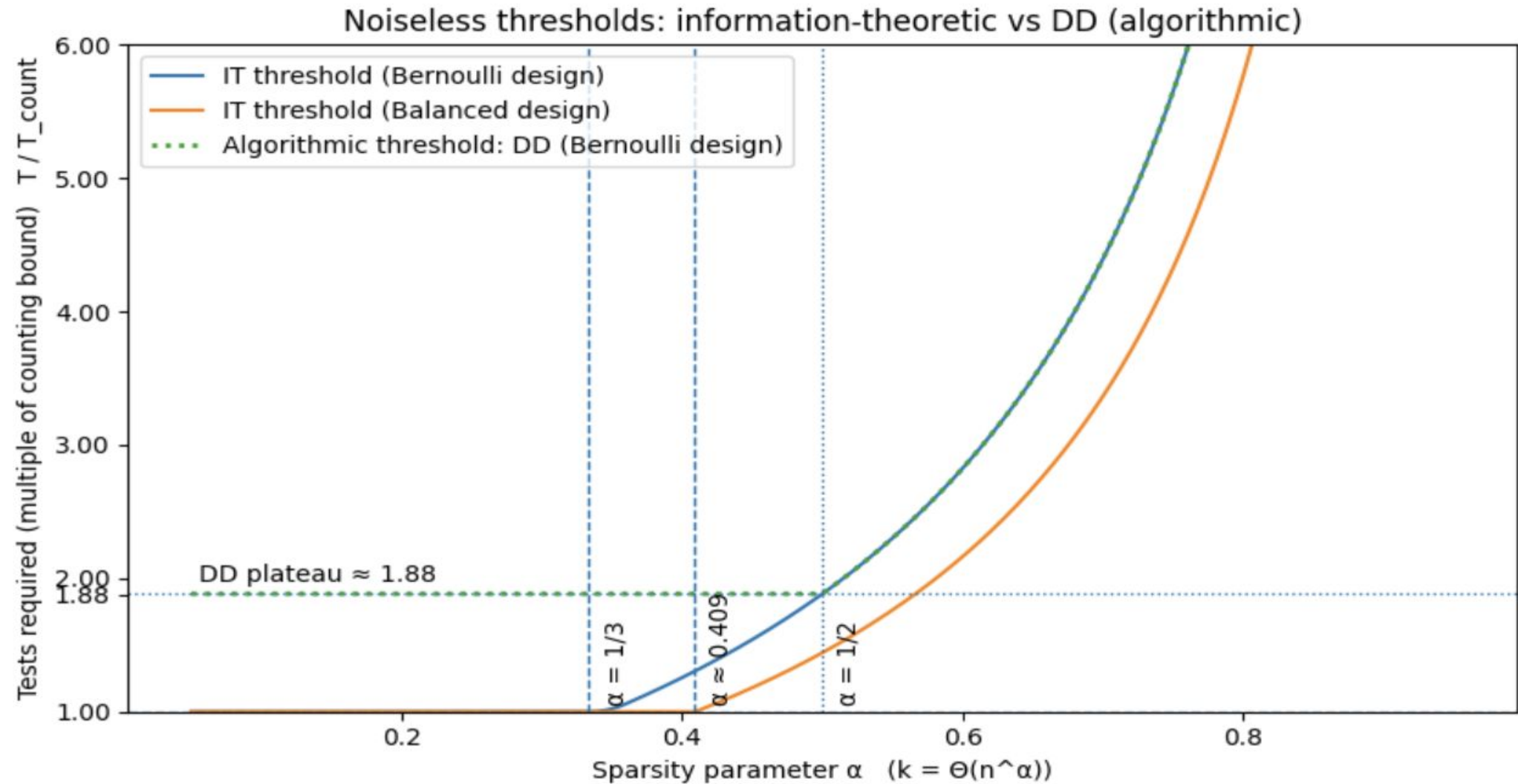


The counting bound marks a fundamental information limit; meaningful recovery thresholds exist only above it.

Key question:

When can efficient algorithms match the information-theoretic limit?





The bottleneck is isolation, not elimination.

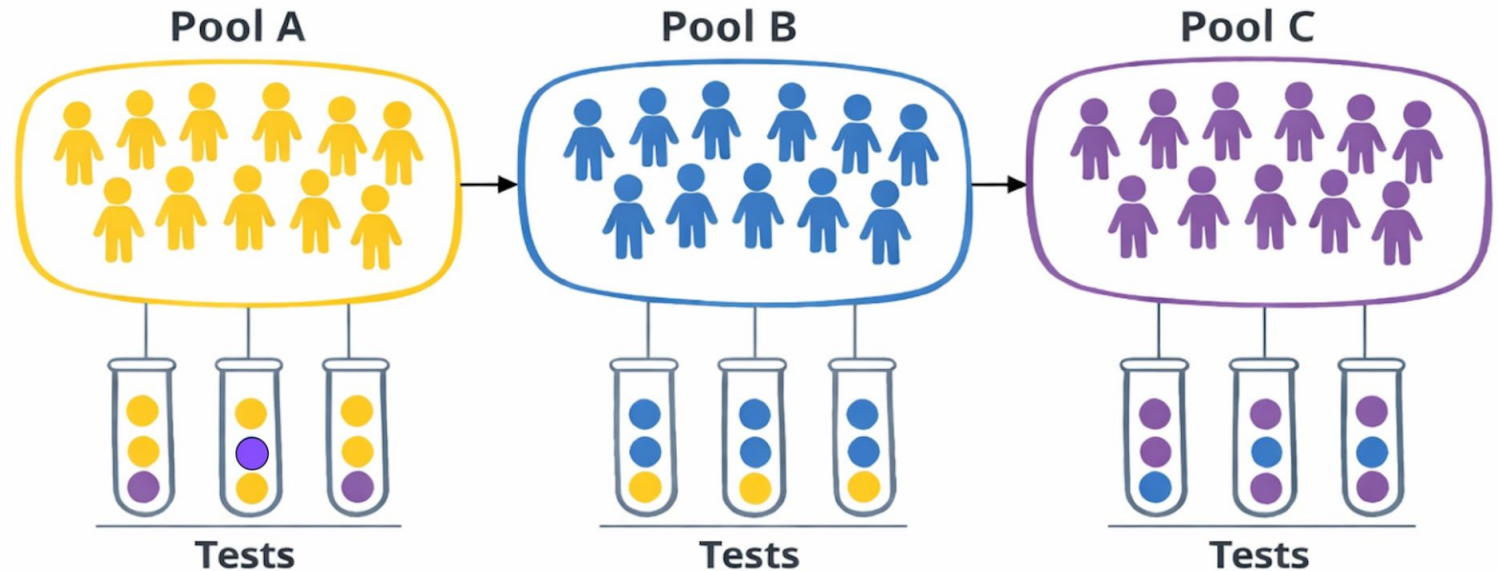
- Algorithms like DD succeed only when defectives can be **locally isolated**
- In **dense regimes**, isolation occurs naturally
- In **sparse regimes**, defectives are masked by surviving candidates

Result:

Even when enough information exists globally, local isolation may not occur → algorithmic gap.

Idea: Change the *test design*, not the algorithm.

- Individuals are split into **ordered pools**
- Tests are random **within pools**, overlapping across neighbors
- Enables **local randomness with global information flow**



Spatially coupled group testing design.

Key effect:

- Early pools become identifiable first
- Their information **propagates forward**, enabling isolation everywhere
- Simple algorithms (like DD) now achieve **near information-theoretic limits**

- From wartime screening to modern data-driven systems
- A fundamental problem of **finding rare signals efficiently**
- Appears across biology, networks, and data science
- **Key challenge:** designing tests that balance randomness and structure
- Many limits still unknown — theory and algorithms continue to evolve

Thank You!

TUHH

Questions?